MERN STACK

TRAINING TR-103 REPORT DAY 21

18 July 2026

Overview:

Today I started learning MongoDB, a NoSQL database used for storing data in a flexible, document-based format. I learned about the features of MongoDB, its installation process, and how to connect it with a Node.js application using Mongoose. The session also covered setting up a backend project with Express.js, CORS, and MongoDB to establish communication between the frontend and the database.

Learning Objectives:

- Understand the basics of MongoDB.
- Learn the features of MongoDB.
- Install and configure MongoDB.
- Set up a backend using Express.js.
- Connect MongoDB with Node.js using Mongoose.
- Understand the purpose of CORS in web applications

MongoDB

MongoDB is a NoSQL, document-oriented database that stores data in flexible BSON (Binary JSON) documents instead of tables with rows and columns. It is widely used in MERN Stack applications because it can efficiently handle large amounts of structured and unstructured data.

Features:

- Document-oriented database.
- Stores data in BSON format.
- Schema-less structure.
- High performance.
- Supports horizontal scaling.
- High availability using replica sets.
- Supports indexing and aggregation.

MongoDB Installation

MongoDB can be installed by downloading the MongoDB Community Server from the official website and following the installation steps.

Installation Steps:

- Download MongoDB Community Server.
- Install using the Complete Setup option.
- Install MongoDB Compass (optional).
- Verify installation using:

```
mongod --version
```

Or

```
mongosh
```

Backend Setup

A backend application was created using Node.js, Express.js, and MongoDB to build REST APIs and connect them with the database.

Steps:

- Create a backend folder.
- Initialize the project using:

```
npm init -y
```

- Install required packages:

```
npm install express mongoose cors
```

Express.js

Express.js is a lightweight Node.js framework used to build web servers and REST APIs quickly.

Features:

- Simplifies backend development.
- Handles HTTP requests and responses.
- Supports middleware.
- Makes API development easier.

Mongoose

Mongoose is an Object Data Modeling (ODM) library that connects Node.js applications with MongoDB. It provides schemas and models to organize and validate data.

Features:

- Connects Node.js with MongoDB.
- Defines schemas and models.
- Performs CRUD operations.
- Validates data before storing it.

Schema

A Schema defines the structure of documents stored in a MongoDB collection. It specifies the fields, their data types, and validation rules, ensuring that data is stored in a consistent format.

Features:

- Defines document structure.
- Specifies field names and data types.
- Supports validation rules.
- Used to create Mongoose models.

Example:

```
const mongoose = require("mongoose");


const StudentSchema = new mongoose.Schema({
  name: String,
  age: Number,
  email: String
});


module.exports = mongoose.model("Student", StudentSchema);
```

CORS

CORS (Cross-Origin Resource Sharing) is a security mechanism that allows a frontend and backend running on different origins (ports or domains) to communicate with each other.

Features:

- Enables frontend-backend communication.
- Prevents unauthorized cross-origin requests.
- Commonly used in MERN applications.

Server Configuration

The Express server was configured by importing the required modules, enabling middleware, connecting to MongoDB, and starting the server.

Main Components:

- Import Express, Mongoose, and CORS.
- Create an Express application.

- Enable JSON parsing.
- Connect to MongoDB.
- Start the server using app.listen().

Task –1

```
const mongoose = require("mongoose");          // for db
const express  = require("express");           //To connet db & frontend
const cors = require("cors")                   //frontend and backend
const app = express()

app.use(cors())
app.use(express.json())

app.listen(9000,() =>{
    console.log("Server is Runnning")

})

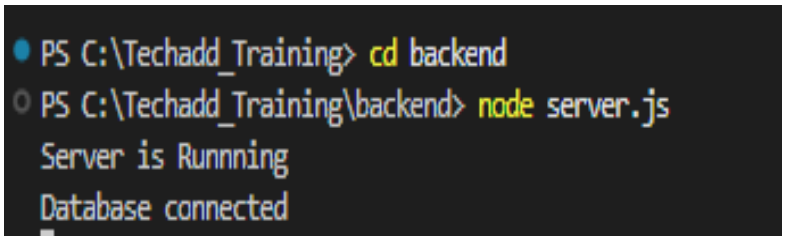
mongoose.connect("mongodb://127.0.0.1:27017/Backend")
.then(()=>{
    console.log("Database connected")
})

.catch((err)=>{
    console.log(err)
})

const userSchema = mongoose.Schema({
    Name: String,
    Email : String,
    Age : Number
})

const usermodel = mongoose.model("Detail",userSchema,"Detail")
```

Output:



Conclusion:

By the end of Day 21, I gained a solid understanding of MongoDB and backend setup in the MERN Stack. I learned how to install MongoDB, configure an Express server, connect it to MongoDB using Mongoose, create schemas for data modeling, and enable communication between the frontend and backend using CORS.